

INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE ESTRUTURAS DE DADOS
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

Estruturas de Dados

Aula 8 - ED - Filas Sequenciais Circulares

Introdução

Olá,

Na semana anterior nós implementamos a Estrutura de Dados Fila, tomando por base uma Lista Sequencial e Lista Simplesmente Encadeada. Duas implementações!

Quando tratamos da implementação sequencial, foi comentado sobre o custo computacional relacionado à operação dequeue. Se você bem se lembra, tal operação tinha um custo computacional elevado porque arrastava todos os demais elementos da Fila em cada remoção (dequeue).

Hoje veremos a mesma Estrutura de Dados, a **Fila**, porém com uma implementação sequencial melhorada. Em essência, faremos alguns ajustes na implementação sequencial para tornar a operação dequeue mais eficiente. A estratégia é considerar o array como tendo forma circular, com o fim sendo seguido do início, razão pela qual a estrutura passa a ser conhecida por **Fila Sequencial Circular**.

“Pega um mói de atenção e vai...”

Filas

Como já sabemos, uma **Fila** é uma estrutura que armazena elementos do mesmo tipo, em que o primeiro elemento inserido deve ser o primeiro a ser retirado. Por essa razão, a Estrutura Fila é conhecida pelo termo FIFO (*First In First Out* - Primeiro a Entrar é o Primeiro a Sair). Podemos enxergar uma Fila como uma Lista que sempre insere um novo elemento no fim e sempre obtém um elemento do início.

Na aula de hoje iremos implementar uma Fila usando a ideia de um array circular. Um array circular é nada mais nada menos que um array onde consideramos o fim seguido imediatamente do início. Em função disso, não há mais a necessidade de puxarmos os elementos quando executarmos a operação dequeue, mas tão somente avançar o índice do array que controla o início da Fila. Agora teremos uma variável que controla o início e outra que controla o fim. Se um elemento é inserido na Fila, o índice de fim

caminha para frente. Se, por outro lado, um elemento é removido da Fila, o índice de início caminha para frente. Observe a imagem a seguir.

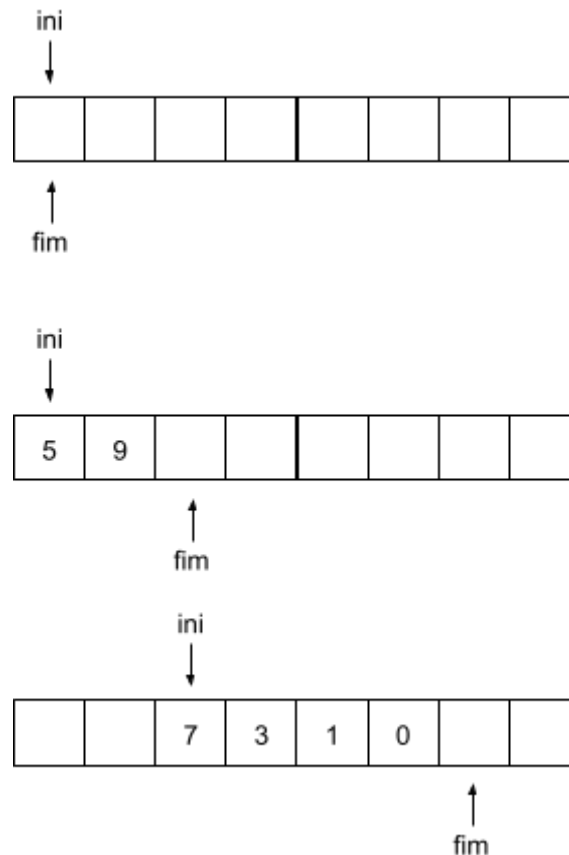


Imagem 1 - Exemplo funcionamento índices ini e fim.

Na imagem acima temos, primeiramente, a situação de uma Lista vazia. Depois, está ilustrado a inserção de dois elementos na Fila. Note que o índice de fim sempre está um passo à frente. Na sequência, mais alguns elementos foram inseridos e os dois primeiros elementos foram removidos. Note como os índices ini e fim se posicionam.

Ainda pensando em termos do exemplo acima, e se quisermos inserir mais três elementos na Fila? Como seria possível, se o índice de fim está a dois passos do limite do array? É simples! Se o índice de fim alcançar o limite do array, basta avançá-lo para o início do array, caso o espaço não esteja ocupado. Veja:

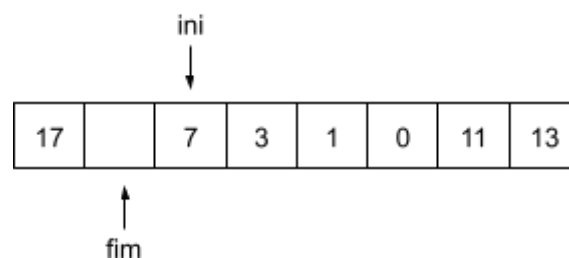


Imagem 2 - Exemplo funcionamento array circular.

Agora temos a situação em que o índice de fim está atrás do índice de início. Considerando a ordem de inserção dos elementos na Estrutura, temos que o elemento 7 é o primeiro elemento e o elemento 17, o último elemento.

Note que, apesar de ainda haver um espaço no array, a Estrutura em si está cheia. Nessa implementação sempre deverá existir um espaço vazio entre os índices de fim e de ini. Sempre que o índice de fim estiver a uma passo do índice de ini, quer dizer que a Estrutura está cheia. E por que não podemos aproveitar o espaço vazio? Porque se permitirmos mais um elemento na Fila, o índice fim avançaria para ser igual ao índice ini. E se isso acontecer, voltaríamos à situação de início da Fila, quando os índices eram iguais, caracterizando uma Fila vazia.

Em relação ao índice ini: se, ao remover um elemento, alcançarmos o fim do array, basta levar o índice ini para o início do array à semelhança do que fizemos com o índice fim. Para exemplificar, vamos remover os elementos 7 à 13. Veja:

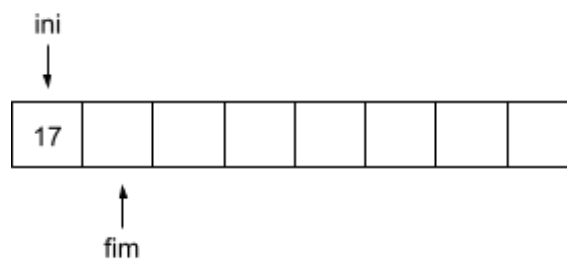


Imagem 3 - Removendo um elemento do fim do array.

Agora temos que o índice ini está de volta ao início do array.

Note que nada do que foi comentado acima deve ser percebido pelo usuário da Fila Sequencial Circular. Tudo acontece de maneira transparente.

Fila Sequencial Circular

Podemos aproveitar a implementação de Fila Sequencial da última aula como base. Todas as funções sofrerão modificações.

Código

Segue abaixo o código.

```
#include <stdio.h>
#include <stdlib.h>

const unsigned MAX = 10;
int l[MAX], ini = 0, fim = 0;

void enqueue(int elemento);
```

```

int dequeue();

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void enqueue(int elemento){
    if ((fim + 1) != ini && !(fim + 1 == MAX && !ini) ){
        l[fim] = elemento;
        //fim = (fim + 1) % MAX;
        fim++;
        if (fim == MAX) fim = 0;
    }
    else
        printf("Não foi possível inserir %d. Fila
cheia.\n",elemento);
}

int dequeue(){
    if (ini != fim) {
        int dado = l[ini];
        //ini = (ini + 1) % MAX;
        ini++;
        if (ini == MAX) ini = 0;
        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
        exit(1);
    }
}

int tamanho(){
    if (ini <= fim)
        return fim - ini;
    else
        return MAX - (ini - fim);
}

```

```

}

void imprimir(){
    printf("F[ ");
    for (int i = ini ; ; i++) {
        if (i == MAX) i = 0;
        if (i == fim) break;
        printf("%d ", l[i]);
    }
    printf("]\n");
}

void apagar(){
    ini = fim = 0;
}

```

Como exercício inicial, analise o código e assinale as diferenças da Fila Sequencial da aula passada. Vamos precisar de:

- algumas variáveis e uma constante:
 - uma constante **MAX**, para definir o tamanho máximo da Fila
 - um array **l**, para armazenar os elementos da Fila
 - uma variável **fim**, para controlar a inserção (enqueue) de elementos na Fila.
 - uma nova variável **ini**, para controlar a remoção (dequeue) de elementos na Fila.
- duas funções base:
 - **enqueue**, para enfileirar um novo elemento na Fila
 - **dequeue**, para desenfileirar o primeiro elemento da Fila
- algumas funções auxiliares:
 - **tamanho**, para obter o tamanho da Fila
 - **imprimir**, para exibir toda a Fila
 - **apagar**, para zerar a Fila

Vamos entender como cada função funciona.

Funções base

Começaremos pelas funções base da Estrutura.

Função enqueue

A função enqueue tem o objetivo de inserir um elemento na Fila.

```

void enqueue(int elemento){
    if ((fim + 1) != ini && !(fim + 1 == MAX && !ini) ){

```

```

        l[fim] = elemento;
        fim++;
        if (fim == MAX) fim = 0;
    }
    else
        printf("Não foi possível inserir %d. Fila
cheia.\n", elemento);
}

```

A função não retorna nada e recebe o elemento a ser inserido. Primeiramente, verificamos se ainda há espaço para novos elementos. Na implementação circular, existem duas situações que caracterizam a Fila como cheia: o índice $fim + 1 == ini$ ou o índice $fim + 1 == MAX$ e $ini == 0$. Portanto, **se no momento de inserir, não caímos na primeira situação nem a segunda**, então há espaço para o novo elemento. Atribuímos elemento ao índice fim e incrementamos o índice fim . Em seguida, verificamos se fim alcançou MAX . Se sim, levamos fim para o início do array.

Caso a Fila esteja cheia, informamos ao usuário.

Função dequeue

A função dequeue tem o objetivo de retornar o primeiro elemento da Fila.

```

int dequeue(){
    if (ini != fim) {
        int dado = l[ini];
        ini++;
        if (ini == MAX) ini = 0;
        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
        exit(1);
    }
}

```

A função retorna o elemento do início da Fila (nesse caso é do tipo inteiro) e não recebe nada. Primeiro verificamos se a Fila não está vazia, avaliando se ini é diferente de fim . Pois caso sejam diferentes, então há elementos na Fila. Em seguida, armazenamos em uma variável temporária o primeiro elemento. Isso é necessário para fazermos alguns ajustes antes de retornar o elemento. Depois incrementamos ini e verificamos se ini alcançou MAX . Se sim, levamos ini para o início do array. Por fim, retornamos o dado.

Caso a Fila esteja vazia, situação em que $ini == fim$, imprimimos o aviso na tela e encerramos o programa. Como a função deve retornar algum elemento, não há como continuar se não houver elemento.

Funções auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Fila.

```
int tamanho(){
    if (ini <= fim)
        return fim - ini;
    else
        return MAX - (ini - fim);
}
```

Nesta implementação de Fila Sequencial Circular, a função tamanho ficou levemente mais complexa. Para calcular a quantidade de elementos (tamanho da Fila), precisamos primeiro saber se ini é menor ou igual a fim. Se sim, o cálculo é trivial: $fim - ini$ nos dará o tamanho da Fila. Caso contrário, então é porque o índice fim já passou do limite do array e voltou ao início. Nessa situação, temos que subtrair de MAX a diferença entre ini e fim.

Função imprimir

A função imprimir não recebe nem retorna nada. Apenas imprime o conteúdo da Fila na tela.

```
void imprimir(){
    printf("F[ ");
    for (int i = ini ; ; i++) {
        if (i == MAX) i = 0;
        if (i == fim) break;
        printf("%d ", l[i]);
    }
    printf("]\n");
}
```

Aqui, fazemos o contador i partir de ini e correr até alcançar o fim. Dentro do loop, verificamos se i alcançou MAX e ajustamos para zero. Em cada loop, imprimimos o respectivo elemento da Fila.

Função apagar

A função apagar não recebe nem retorna nada. Tem o objetivo de zerar a Fila.

```
void apagar(){  
    ini = fim = 0;  
}
```

Para apagar a Fila Sequencial Circular, basta zerar os índices ini e fim.

Por fim, devemos observar que, do ponto de vista de quem usa a Fila Sequencial Circular, não existem diferenças entre ela e as versões implementadas anteriormente. Claro, desprezando o limite máximo de elementos imposto pela implementação sequencial. O grande diferencial das Filas Sequenciais Circulares em relação às demais diz respeito à otimização nas operações enqueue e dequeue. Em relação à implementação sequencial anterior, que puxa os elementos, reduzimos a complexidade. Em relação à implementação com Listas Simplesmente Encadeadas, uma vez que temos o array, poupamos a custosa operação de alocação e desalocação de memória.

Na aula de hoje demos uma nova roupagem à implementação de Filas, chegando às Filas Sequenciais Circulares.

Tragam as sementes para a próxima aula, pois iremos trabalhar com a Estrutura Árvore.



Bons estudos e até a próxima!